



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Дорогие друзья, в этом разделе мы повысим эффективность работы с файлами и текстом, познакомившись с несколькими командами: `grep`, `sed`, `cut` и `awk`. Расширенное использование некоторых из этих инструментов требует хорошего понимания работы регулярных выражений (`regex`). Регулярное выражение – это специальный шаблон, по которому будет производиться поиск. Если вы не знакомы с регулярными выражениями, посетите следующие ресурсы, прежде чем продолжить:

<http://www.rexegg.com> <http://www.regular-expressions.info>

grep

В двух словах, `grep` ищет в текстовых файлах вхождения заданного регулярного выражения и выводит любую строку, содержащую совпадение, на стандартный вывод, который обычно является экраном терминала. Некоторые из наиболее часто используемые параметров: `-r` для рекурсивного поиска и `-i` для игнорирования регистра текста. Рассмотрим следующий пример:

```
kali@kali:~$ ls -la /usr/bin | grep zip
-rwxr-xr-x 3 root root 34480 Jan 29 2017 bunzip2
-rwxr-xr-x 3 root root 34480 Jan 29 2017 bzip2
-rwxr-xr-x 1 root root 13864 Jan 29 2017 bzip2recover
...
```

P.S. Ключ `l` - вывод в длинном формате (режим доступа и т.д.), ключ `a` - вывод всех файлов, включая скрытые.

Мы перечислили все файлы в каталоге `/usr/bin` с помощью `ls` и передали вывод в команду `grep`, которая ищет любую строку, содержащую строку `"zip"`. Понимание инструмента `grep` и того, когда его использовать, может оказаться невероятно полезным.

`grep` поддерживает регулярные выражения, что позволяет выполнять мощный и гибкий поиск по шаблону. Например, команда `grep "error" logfile.txt` выведет все строки из файла `logfile.txt`, содержащие слово `"error"`. Кроме того, `grep` имеет множество опций, таких как `-i` для игнорирования регистра, `-r` для рекурсивного поиска в каталогах, и `-v` для инвертирования поиска, чтобы показать строки, не соответствующие шаблону.

sed

sed – это мощный редактор потоков. Он также очень сложен, поэтому здесь я лишь вкратце опишу его возможности. На очень высоком уровне sed выполняет редактирование текста в потоке, либо в наборе определенных файлов, либо в стандартном выводе. Давайте рассмотрим пример:

```
kali@kali:~$ echo "I need to try hard" | sed 's/hard/harder/'
```

```
I need to try harder
```

Мы создали поток текста с помощью команды echo, а затем передали его в sed, чтобы заменить слово "hard" на "harder" (символ s - это команда замены).

cut

Команда cut проста, но часто оказывается весьма полезной. Она используется для выделения фрагмента текста из строки и вывода его на стандартный вывод. Некоторые из наиболее часто используемых параметров: -f для номера поля, которое мы выделяем, и -d для разделителя полей.

```
kali@kali:~$ echo "a,b,c,d" | cut -f 2 -d ","
```

```
awk
```

AWK – это, по сути, язык программирования, предназначенный для обработки текста и обычно используемый в качестве инструмента извлечения данных и составления отчетов. Он также чрезвычайно мощный и может быть довольно сложным в понимании, поэтому здесь я лишь поверхностно рассмотрю его. В awk часто используется параметр -F, который является разделителем полей, и команда print, которая выводит текст результата.

```
kali@kali:~$ echo "hello::my::friend" | awk -F "::" '{print $1, $3}'
```

Мы передали строку в awk для извлечения первого (\$1) и третьего (\$3) полей, используя :: в качестве разделителя полей.

Практический пример

Давайте рассмотрим практический пример, который связывает воедино многие из команд, которые мы изучили. Нам дан журнал HTTP-сервера Apache (http://www.offensive-security.com/pwk-files/access_log.txt.gz), который содержит доказательства атаки. Наша задача - с помощью команд Bash изучить этот файл и обнаружить различные фрагменты информации, например, кто был атакующим и что именно произошло на сервере.

Сначала мы воспользуемся командами `head` и `wc`, чтобы бегло просмотреть файл журнала и понять его структуру. Команда `head` отображает первые 10 строк в файле, а команда `wc`, вместе с опцией `-l`, отображает общее количество строк в файле.

```
kali@kali:~$ gunzip access_log.txt.gz
kali@kali:~$ mv access_log.txt access.txt
kali@kali:~$ head access.txt
kali@kali:~$ wc -l access.txt
```

Обратите внимание, что файл журнала является текстовым и содержит различные поля (IP-адрес, временная метка, HTTP-запрос и т.д.), которые разделены пробелами. Это совершенно "дружественный" файл для команды `grep`, и он будет хорошо работать со всеми инструментами, которые мы рассмотрели до сих пор. Мы начнем с поиска HTTP-запросов, сделанных к серверу, записанных в этом файле журнала. Для этого мы направим вывод команды `cat` в команды `cut` и `sort`. Это может дать нам представление о количестве потенциальных злоумышленников, с которыми нам придется иметь дело.

```
kali@kali:~$ cat access.txt | cut -d " " -f 1 | sort -u
```

Мы видим, что менее десяти IP-адресов были зарегистрированы в журнале, хотя это все равно ничего не говорит нам о злоумышленниках. Далее мы используем команды `uniq` и `sort` для отображения уникальных строк, и отсортируем данные по количеству обращений к серверу каждого IP-адреса. Опция `-c` в `uniq` добавит в выходную строку префикс с количеством вхождений.

```
kali@kali:~$ cat access.txt | cut -d " " -f 1 | sort | uniq -c | sort -urn
```

- n - сортировка строк Linux по числовому значению
- r - сортировать в обратном порядке
- u - игнорировать повторяющиеся строки

Выделяется несколько IP-адресов, но мы сосредоточимся на адресе, который имеет самую высокую частоту обращений. Чтобы отфильтровать адрес 208.68.234.99, отобразить и подсчитать ресурсы, которые запрашивались с этого IP, мы можем использовать следующую последовательность:

```
kali@kali:~$ cat access.txt | grep '208.68.234.99' | cut -d "\"" -f 2 | uniq -c
```

где "\"" – это разделитель!

Из вывода видно, что IP-адрес 208.68.234.99 обращался исключительно к каталогу /admin. Давайте проверим это:

```
kali@kali:~$ cat access.txt | grep '208.68.234.99' | grep '/admin' | sort -u
```

Очевидно, что 208.68.234.99 был вовлечен в попытку перебора пароля против этого веб-сервера. Более того, после примерно 1000 попыток, похоже, что попытка перебора увенчалась успехом, о чем свидетельствует сообщение "HTTP 200".

Команды, используемые в уроке:

```
kali@kali:~$ ls -la /usr/bin | grep zip
```

```
kali@kali:~$ echo "I need to try hard" | sed 's/hard/harder/'
```

```
kali@kali:~$ echo "a,b,c,d" | cut -f 2 -d ","
```

```
kali@kali:~$ echo "hello::my::friend" | awk -F "::" '{print $1, $3}'
```

```
kali@kali:~$ gunzip access_log.txt.gz
```

```
kali@kali:~$ mv access_log.txt access.txt
```

```
kali@kali:~$ head access.txt
```

```
kali@kali:~$ wc -l access.txt
```

```
kali@kali:~$ cat access.txt | cut -d " " -f 1 | sort -u
```

```
kali@kali:~$ cat access.txt | cut -d " " -f 1 | sort | uniq -c | sort -urn
```

```
kali@kali:~$ cat access.txt | grep '208.68.234.99' | cut -d "\"" -f 2 | uniq -c
```

```
kali@kali:~$ cat access.txt | grep '208.68.234,99' | grep '/admin' | sort -u
```